Q 1 : Implement Union, Intersection, Complement, Algebraic Difference, Algebraic product, Multiplication of fuzzy set with crisp number, Power of fuzzy set, Algebraic sum, Bounded sum, Bounded difference and Cartesian product on fuzzy sets.

UNION:

```
[ ]  set1=[0.2,0.4,0.9,0.3]
     set2=[0.4,0.5,0.9,0.6]
```

```
[ ]  def fuzzy_union(set1, set2):
         union=[]
         for i in range(len(set1)):
             union.append(max(set1[i],set2[i]))
         return union
```

```
     union=fuzzy_union(set1, set2)
     print(union)
```

```
     [0.4, 0.5, 0.9, 0.6]
```

INTERSECTION:

```
     set1=[0.2,0.4,0.9,0.3]
     set2=[0.4,0.5,0.9,0.6]
```

```
[ ]  def fuzzy_intersection(set1, set2):
         intersection=[]
         for i in range(len(set1)):
             intersection.append(min(set1[i],set2[i]))
         return intersection
```

```
[ ]  intersection=fuzzy_intersection(set1, set2)
     print(intersection)
```

```
     [0.2, 0.4, 0.9, 0.3]
```

COMPLEMENT:

```
set1=[0.2,0.4,0.9,0.3]
set2=[0.4,0.5,0.9,0.6]
```

```
def fuzzy_complement(set1):
    complement=[]
    for i in range(len(set1)):
        complement.append(1-set1[i])
    return complement
```

```
complement=fuzzy_complement(set1)
print(complement)
```

```
[0.8, 0.6, 0.09999999999999998, 0.7]
```

ALGEBRAIC DIFFERENCE:

```
def difference(a,b):
  res=[(fuzzy_intersection(a,fuzzy_complement(b)))]
  return res
a=[0.2,0.8,0.1]
b=[0.5,0.4,0.2]
print(difference(a,b))
```

```
[[0.2, 0.6, 0.1]]
```

ALGEBRAIC PRODUCT:

```
def fuzzy_product(set1, set2):
    product=[]
    for i in range(len(set1)):
        product.append(set1[i]*set2[i])
    return product
```

```
set1=[0.2,0.4,0.9,0.3]
set2=[0.4,0.5,0.9,0.6]

product = fuzzy_product(set1, set2)
print(product)
```

```
[0.08000000000000002, 0.2, 0.81, 0.18]
```

MULTIPLICATION OF FUZZY SET WITH CRISP NUMBERS:

```
def scalar_product(alpha,a):
  res=[alpha*a[i] for i in range(len(a))]
  return res
a=[0.1,0.8,0.1]
print(scalar_product(2,a))
```

```
[0.2, 1.6, 0.2]
```

POWER OF FUZZY SET:

```
import itertools

def power_set(s):
  return list(itertools.chain.from_iterable(itertools.combinations(s,r) for r in range(len(s)+1)))
s=[0.5,0.4,0.2]
print(power_set(s))
```

```
[(), (0.5,), (0.4,), (0.2,), (0.5, 0.4), (0.5, 0.2), (0.4, 0.2), (0.5, 0.4, 0.2)]
```

ALGEBRAIC SUM:

```
set1=[0.2,0.4,0.9,0.3]
set2=[0.4,0.5,0.9,0.6]
```

```
def fuzzy_sum(set1, set2):
    sum=[]
    for i in range(len(set1)):
        sum.append((set1[i]+set2[i]) - set1[i]*set2[i])
    return sum
```

```
sum = fuzzy_sum(set1, set2)
print(sum)
```

```
[0.52, 0.7, 0.99, 0.72]
```

BOUNDED SUM:

```
[ ]  def bounded_sum(a,b):
       res=[min(1,((a[i]+b[i]))) for i in range(len(a))]
       return res
     a=[0.2,0.8,0.1]
     b=[0.5,0.4,0.2]
     print(bounded_sum(a,b))
```

```
[0.7, 1, 0.30000000000000004]
```

BOUNDED DIFFERENCE:

```
[ ]  def bounded_diff(a,b):
       res=[max(0,((a[i]+b[i])-1)) for i in range(len(a))]
       return res
     a=[0.2,0.8,0.1]
     b=[0.5,0.4,0.2]
     print(bounded_diff(a,b))
```

```
[0, 0.20000000000000018, 0]
```

CARTESIAN PRODUCT:

```
import numpy as np
def cartesian_product(a,b):
  k=[]
  for i in range(len(a)):
    res=[min(a[i],b[j]) for j in range(len(b))]
    k.append(res)
  return k
a=[0.2,0.3,0.5,0.6]
b=[0.8,0.6,0.3]
print(np.array(cartesian_product(a,b)))
```

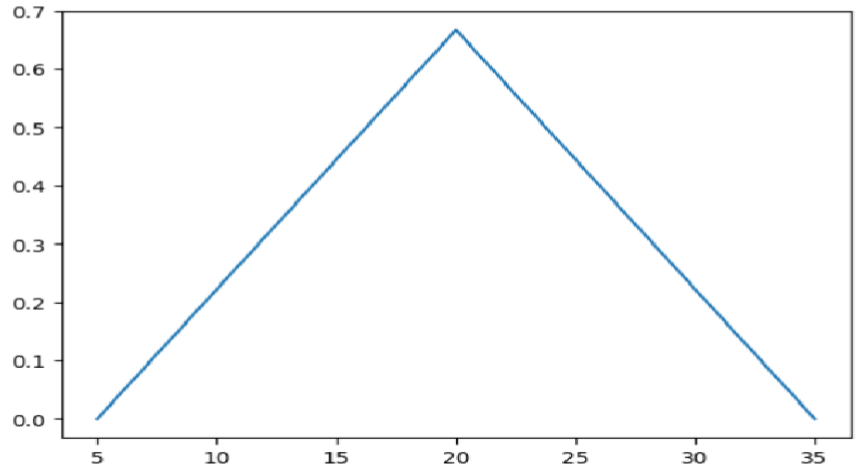
```
[[0.2 0.2 0.2]
 [0.3 0.3 0.3]
 [0.5 0.5 0.3]
 [0.6 0.6 0.3]]
```

Q2 : Realization and plotting of different fuzzy membership function: Triangular Membership function, Trapezoidal membership function, Sigmoid membership function, Generalized bell membership function and Gaussian membership function.

TRIANGULAR MEMBERSHIP FUNCTION:

```
[ ]  import matplotlib.pyplot as plt
     def triangle(x,a,b,c):
       return max(min((x-a)/(b-a),(c-x)/(c-b)),0)

     x=triangle(15,5,20,35)
     print(x)
     plt.plot([5,20,35],[0,x,0])
     plt.show()
```

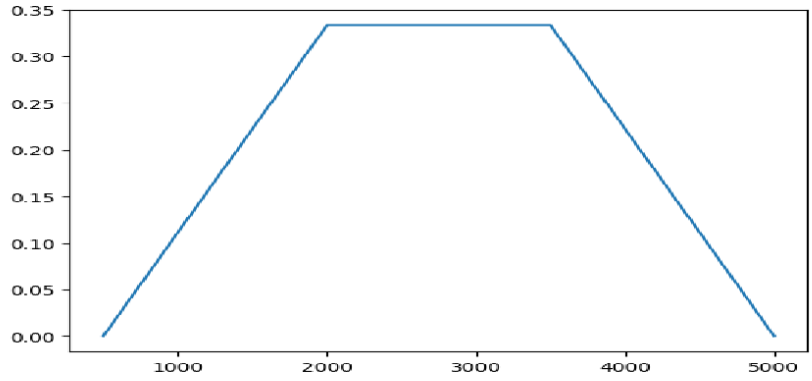
```
0.6666666666666666
```



TRAPEZOIDAL MEMBERSHIP FUNCTION:

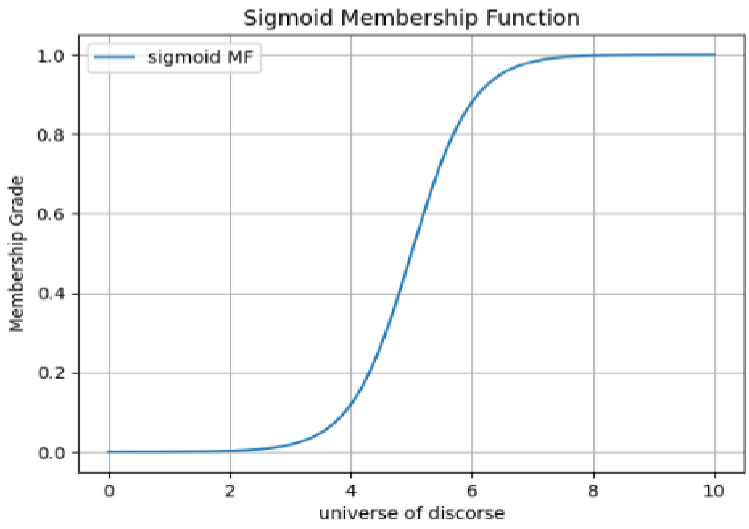
```
import matplotlib.pyplot as plt
def trapezoidal(x,a,b,c,d):
  print((x-a)/(b-a))
  print((d-x)/(d-c))
  return max(min((x-a)/(b-a),1,(d-x)/(d-c)),0)

x=trapezoidal(1000,500,2000,3500,5000)
print(x)
plt.plot([500,2000,3500,5000],[0,x,x,0])
plt.show()
```

```
0.3333333333333333
2.6666666666666665
0.3333333333333333
```



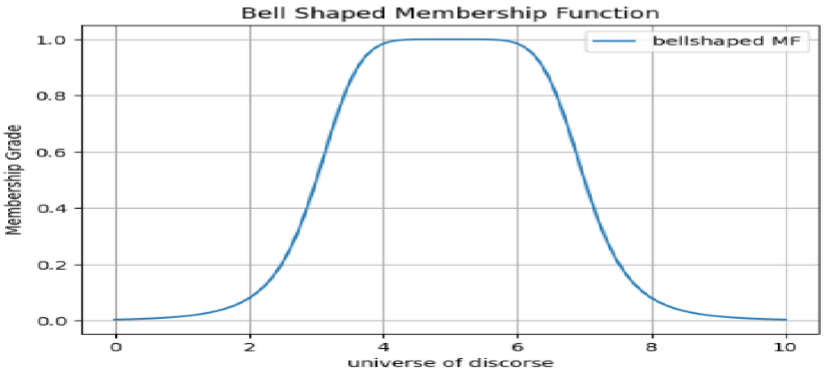
SIGMOID MEMBERSHIP FUNCTION:

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid_mf(x,a,c):
    return 1/(1+ np.exp(-a *(x-c)))

a=2
c=5
x = np.linspace(0, 10, 100)


membership_values = sigmoid_mf(x,a,c)


plt.plot(x, membership_values, label='sigmoid MF')
plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Sigmoid Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



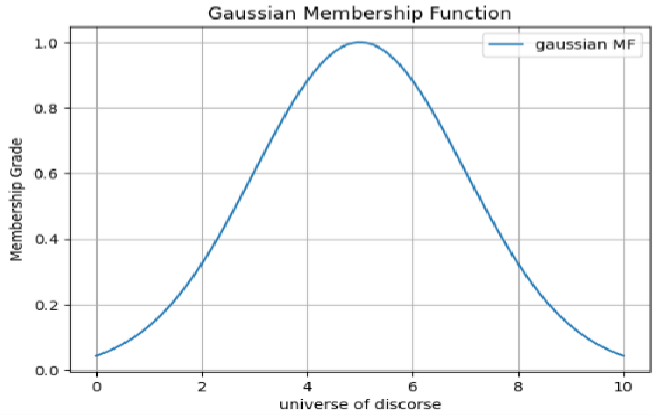
GENERALIZED BELL MEMBERSHIP FUNCTION:

```
import matplotlib.pyplot as plt

def bell_mf(x, a,b,c):
    return 1/(1+ ((x-c)/a) ** (2*b))


a=2
b=3
c=5
x = np.linspace(0, 10, 100)


membership_values = bell_mf(x,a,b,c)


plt.plot(x, membership_values, label='bellshaped MF')
plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Bell Shaped Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



GAUSSIAN MEMBERSHIP FUNCTION:

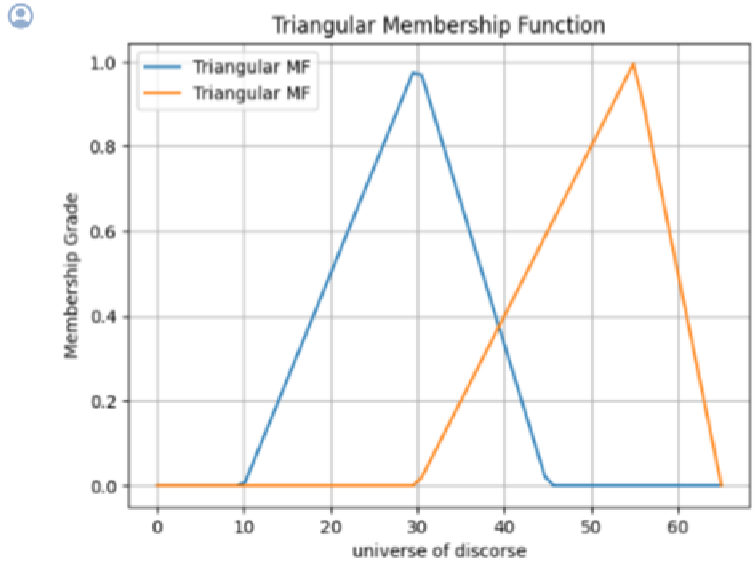
```
import numpy as np
import matplotlib.pyplot as plt

def gaussian_mf(x, mean,sigma):
    return np.exp(-0.5 * ((x-mean)/sigma)**2)


mean=5
sigma=2

x = np.linspace(0, 10, 100)


membership_values = gaussian_mf(x,mean,sigma)


plt.plot(x, membership_values, label='gaussian MF')
plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Gaussian Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



Q3 : Implement fuzzification on crisp set using following membership functions: i. Triangular Membership function ii. Trapezoidal membership function iii. Sigmoid membership function iv. Generalized bell membership function v. Gaussian membership function.

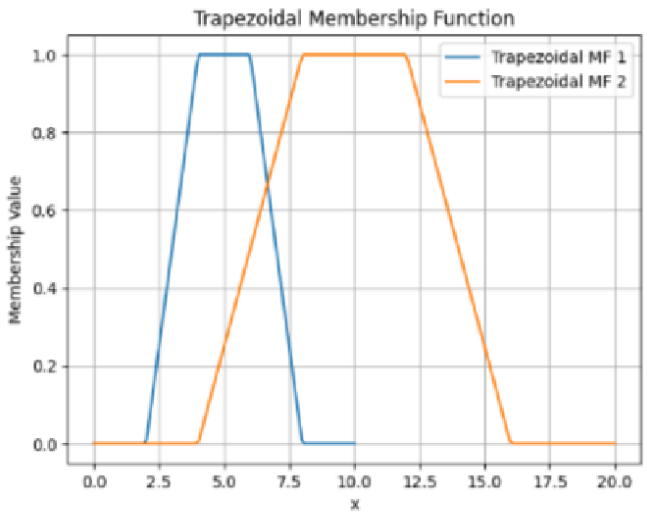
TRIANGULAR MEMBERSHIP FUNCTION:

```
import numpy as np
import matplotlib.pyplot as plt

def triangular_mf(x, a, b, c):
    return np.maximum(0, np.minimum((x - a) / (b - a), (c - x) / (c - b)))

#{a,b,c are crisp set1 } and {a1,b1,c1 are crisp set}
a = 10
b = 30
c = 45


a1=30
b1=55
c1=65

x = np.linspace(0, 65, 65)

membership_values1= triangular_mf(x, a, b, c)
membership_values2 = triangular_mf(x, a1, b1, c1)

plt.plot(x, membership_values1, label='Triangular MF')
plt.plot(x, membership_values2, label='Triangular MF')

plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Triangular Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



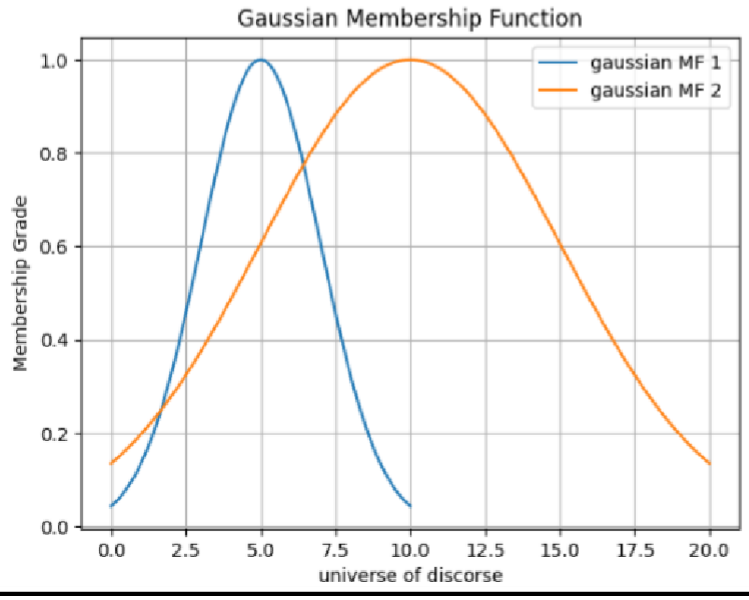
TRAPEZOIDAL MEMBERSHIP FUNCTION:

```
import numpy as np
import matplotlib.pyplot as plt

def trapezoidal_mf(x, a, b, c, d):
    return np.maximum(0, np.minimum(np.minimum((x - a) / (b - a), 1), (d - x) / (d - c)))

#{a,b,c are crisp set1 } and {a1,b1,c1 are crisp set}
a = 2
b = 4
c = 6
d = 8

a1 = 4
b1= 8
c1 = 12
d1 = 16

x = np.linspace(0, 10, 100)
y=np.linspace(0,20,150)

membership_values = trapezoidal_mf(x, a, b, c, d)
membership_values1 = trapezoidal_mf(y, a1, b1, c1, d1)

plt.plot(x, membership_values, label='Trapezoidal MF 1')
plt.plot(y, membership_values1, label='Trapezoidal MF 2')
plt.xlabel('x')
plt.ylabel('Membership Value')
plt.title('Trapezoidal Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



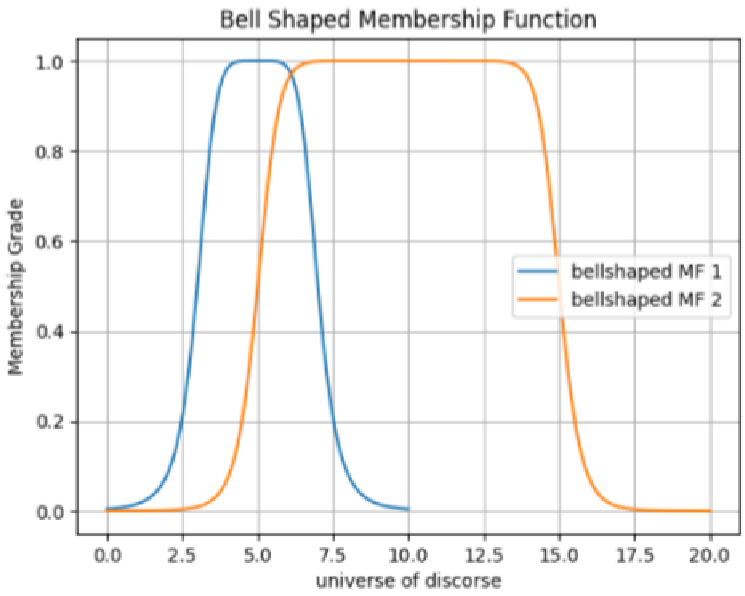
SIGMOID MEMBERSHIP FUNCTION:

```
import numpy as np
import matplotlib.pyplot as plt

def gaussian_mf(x, mean,sigma):
    return np.exp(-0.5 * ((x-mean)/sigma)**2)

#{mean and sigma are for crisp set 1} and {mean1 and sigma1 are for crisp set 2}
mean=5
sigma=2

mean1=10
sigma1=5
x = np.linspace(0, 10, 100)
y = np.linspace(0, 20, 200)

membership_values = gaussian_mf(x,mean,sigma)
membership_values1 = gaussian_mf(y,mean1,sigma1)

plt.plot(x, membership_values, label='gaussian MF 1')
plt.plot(y, membership_values1, label='gaussian MF 2')
plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Gaussian Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



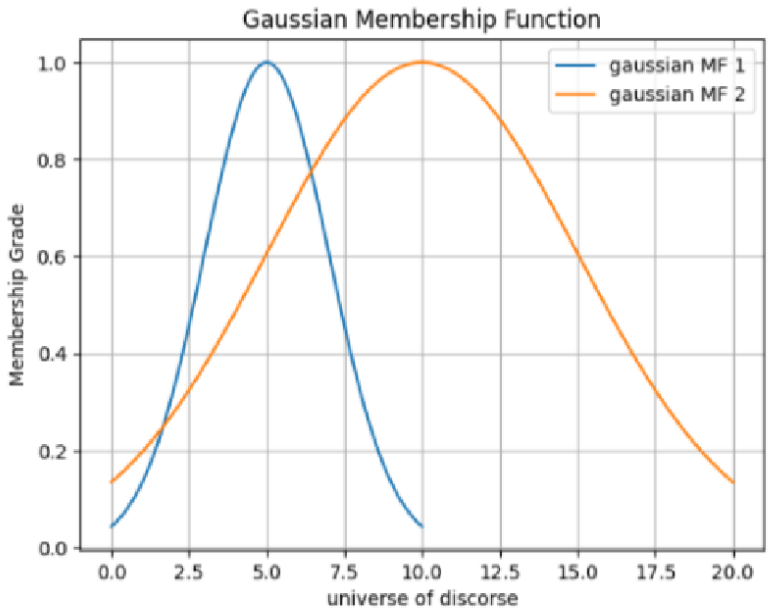
BELL MEMBERSHIP FUNCTION:

```
import numpy as np
import matplotlib.pyplot as plt

def bell_mf(x, a,b,c):
    return 1/(1+ ((x-c)/a) ** (2*b))

#{a,b,c are for crisp set 1 } and {a1,b1,c1 are for crisp set 2}
a=2
b=3
c=5


a1=5
b1=7
c1=10

x = np.linspace(0, 10, 100)
y = np.linspace(0, 20, 200)

membership_values = bell_mf(x,a,b,c)
membership_values1 = bell_mf(y,a1,b1,c1)

plt.plot(x, membership_values, label='bellshaped MF 1')
plt.plot(y, membership_values1, label='bellshaped MF 2')
plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Bell Shaped Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



GAUSSIAN MEMBERSHIP FUNCTION:

```
import numpy as np
import matplotlib.pyplot as plt

def gaussian_mf(x, mean,sigma):
    return np.exp(-0.5 * ((x-mean)/sigma)**2)


mean=5
sigma=2

mean1=10
sigma1=5
x = np.linspace(0, 10, 100)
y = np.linspace(0, 20, 200)

membership_values = gaussian_mf(x,mean,sigma)
membership_values1 = gaussian_mf(y,mean1,sigma1)

plt.plot(x, membership_values, label='gaussian MF 1')
plt.plot(y, membership_values1, label='gaussian MF 2')
plt.xlabel('universe of discorse')
plt.ylabel('Membership Grade')
plt.title('Gaussian Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```



Q4: Realization of characteristics of fuzzy set by finding the following values: i. Core of a fuzzy set ii. Support of a fuzzy set. iii. Crossover points of a fuzzy set. iv. Bandwidth of a fuzzy set. v. Convexity of a fuzzy set.

Core of a fuzzy set:

```
def fuzzy_core(membership_function):
  core = [x for x, mu in membership_function.items() if mu == 1]
  return core
def main():
  membership_function = {}
  num_elements = int(input("Enter the number of elements in the universe of discourse: "))
  print("Enter the name of elements and membership degrees for each element:")
  for i in range(num_elements):
    x = input("Element: ")
    mu = float(input("Membership degree for {}: ".format(x)))
    membership_function[x] = mu
    core = fuzzy_core(membership_function)
    print("Core of the fuzzy set:", core)
main()
```

```
Enter the number of elements in the universe of discourse: 5
Enter the name of elements and membership degrees for each element:
Element: a
Membership degree for a: 0.5
Core of the fuzzy set: []
Element: b
Membership degree for b: 1
Core of the fuzzy set: ['b']
Element: c
Membership degree for c: 4
Core of the fuzzy set: ['b']
Element: d
Membership degree for d: 1
Core of the fuzzy set: ['b', 'd']
Element: e
Membership degree for e: 2
Core of the fuzzy set: ['b', 'd']
```

Support of a fuzzy set:

```
def fuzzy_support(membership_function):
    support = [x for x, mu in membership_function.items() if mu > 0]
    return support
def main():
    membership_function = {}
    num_elements = int(input("Enter the number of elements in the universe of discourse: "))
    print("Enter the name of elements and membership degrees for each element:")
    for i in range(num_elements):
        x = input("Element: ")
        mu= float(input("Membership degree for {}: ".format(x)))
        membership_function[x] = mu
    support = fuzzy_support(membership_function)
    print("Support of the fuzzy set:", support)
main()
```

```
Enter the number of elements in the universe of discourse: 5
Enter the name of elements and membership degrees for each element:
Element: a
Membership degree for a: -.25
Element: b
Membership degree for b: 0
Element: c
Membership degree for c: .5
Element: d
Membership degree for d: 1
Element: e
Membership degree for e: .25
Support of the fuzzy set: ['c', 'd', 'e']
```

Crossover Points of a Fuzzy Set

```
def fuzzy_crossover_points(membership_function):
    crossover_points = [x for x, mu in membership_function.items() if mu == 0.5]
    return crossover_points
def main():
    membership_function = {}
    num_elements = int(input("Enter the number of elements in the universe of discourse: "))
    print("Enter the name of elements and membership degrees for each element:")
    for i in range(num_elements):
        x = input("Element: ")
        mu = float(input("Membership degree for {}: ".format(x)))
        membership_function[x] = mu
    crossover_points = fuzzy_crossover_points(membership_function)
    print("Crossover points of the fuzzy set (membership degree 0.5):", crossover_points)
main()
```

```
Enter the number of elements in the universe of discourse: 5
Enter the name of elements and membership degrees for each element:
Element: a
Membership degree for a: 0.25
Element: b
Membership degree for b: 0.5
Element: c
Membership degree for c: 1
Element: d
Membership degree for d: 0.5
Element: e
Membership degree for e: 0.25
Crossover points of the fuzzy set (membership degree 0.5): ['b', 'd']
```

Bandwidth of a Fuzzy Set

```
def fuzzy_bandwidth(membership_function):
    crossover_points = [float(element) for element, degree in membership_function.items() if degree == 0.5]
    if len(crossover_points) >= 2:
      bandwidth = abs(max(crossover_points) - min(crossover_points))
      return bandwidth
    else:
      return "Bandwidth cannot be calculated. Less than two crossover points found."
def main():
    membership_function = {}
    num_elements = int(input("Enter the number of elements in the universe of discourse: "))
    print("Enter the distance-value of the elements and their membership degrees:")
    for i in range(num_elements):
        element = float(input("Distance of the {}th element: ".format(i+1)))
        degree = float(input("Membership degree for {}th element: ".format(i+1)))
        membership_function[element] = degree
    bandwidth = fuzzy_bandwidth(membership_function)
    print("Bandwidth of the fuzzy set:", bandwidth)
main()
```

```
Enter the number of elements in the universe of discourse: 5
Enter the distance-value of the elements and their membership degrees:
Distance of the 1th element: 25
Membership degree for 1th element: .35
Distance of the 2th element: 50
Membership degree for 2th element: .5
Distance of the 3th element: 75
Membership degree for 3th element: .75
Distance of the 4th element: 92
Membership degree for 4th element: .5
Distance of the 5th element: 110
Membership degree for 5th element: .25
Bandwidth of the fuzzy set: 42.0
```

Convexity of a fuzzy set

```python
def is_convex(membership_function):
    support = [x for x, mu in membership_function.items() if mu > 0]
    if len(support) < 2:
        return False  # Convexity cannot be determined with less than two support points
    for i in range(len(support) - 1):
        x1 = support[i]
        x2 = support[i + 1]
        mid_point = (x1 + x2) / 2
        if mid_point in membership_function:
            mu_mid = membership_function[mid_point]
        else:
            mu_x1 = membership_function[x1]
            mu_x2 = membership_function[x2]
            mu_mid = (mu_x1 + mu_x2) / 2
        mu_x1 = membership_function[x1]
        mu_x2 = membership_function[x2]
        if mu_mid < min(mu_x1, mu_x2):
            return False  # Convexity condition violated for this pair of points
    return True
def main():
    membership_function = {}
    num_elements = int(input("Enter the number of elements in the universe of discourse: "))
    print("Enter the membership degrees for each element:")
    for _ in range(num_elements):
        element = float(input("Element: "))
        degree = float(input("Membership degree for {}: ".format(element)))
        membership_function[element] = degree
    convex = is_convex(membership_function)     # Check convexity of the fuzzy set
    if convex:
        print("The fuzzy set is convex.")
    else:
        print("The fuzzy set is not convex.")
main()
```

```
Enter the number of elements in the universe of discourse: 3
Enter the membership degrees for each element:
Element: 1
Membership degree for 1.0: 0
Element: 2
Membership degree for 2.0: 1
Element: 3
Membership degree for 3.0: 0
The fuzzy set is not convex.
```

Q6: Create fuzzy relation by Cartesian product of any two fuzzy sets and perform max-min and max-product composition on any two fuzzy relations.

```python
def fuzzy_cartesian_product(set1, set2):
 """
 Creates a fuzzy relation by taking the Cartesian product
of two fuzzy sets.
 """
 relation = []
 for a in set1:
   row = []
   for b in set2:
     row.append(min(a, b))
   relation.append(row)
 return relation
```

```python
def fuzzy_max_min_composition(relation1, relation2):

 composition = []
 for i in range(len(relation1)):
   row = []
   for j in range(len(relation2[0])):
      max_min = 0
      for k in range(len(relation1[0])):
         max_min = max(max_min, min(relation1[i][k], relation2[k][j]))

      row.append(max_min)
   composition.append(row)
 return composition

# Example usage
A = [0.2, 0.3, 0.4]
B = [0.4, 0.4 ]
C= [0]
relation1 = fuzzy_cartesian_product(A, B)
relation2 = fuzzy_cartesian_product(B, A)
composition = fuzzy_max_min_composition(relation1,
relation2)
print(f"Relation 1: {relation1}")
print(f"Relation 2: {relation2}")
print(f"Max-min composition: {composition}")
```

```
Relation 1: [[0.2, 0.6, 0.3, 0.7], [0.2, 0.4, 0.3, 0.4], [0.2, 0.6, 0.3, 0.6]]
Relation 2: [[0.2, 0.2, 0.2], [0.6, 0.4, 0.6], [0.3, 0.3, 0.3], [0.7, 0.4, 0.6]]
Max-min composition: [[0.7, 0.4, 0.6], [0.4, 0.4, 0.4], [0.6, 0.4, 0.6]]
```

Q7 Implementation of fuzzy distance matrices: i. Fuzzy Euclidian distance ii. Fuzzy hamming distance

FUZZY EUCLIDIAN DISTANCE:

```
#Fuzzy euclidean distance
def ed(a,b):
  distance = [(i-j)**2 for i,j in zip(a,b)]
  return (sum(distance))**0.5
a=[0.3,0.8,0.1]
b=[0.5,0.7,0.2]
ed(a,b)
```

```
0.24494897427831785
```

HAMMING DISTANCE:

```
#Fuzzy hamming distance
def hd(a,b):
  distance = [(i-j) for i,j in zip(a,b)]
  return abs(sum(distance))
a=[0.3,0.8,0.1]
b=[0.5,0.7,0.2]
hd(a,b)
```

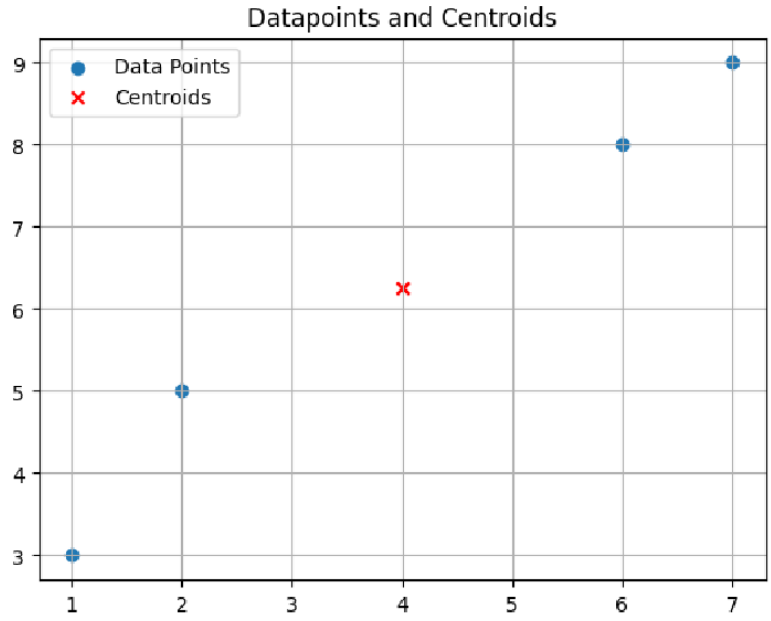
```
0.19999999999999993
```

Q9 Implement a program that performs fuzzy clustering on a set of data points and outputs the membership degrees for each data point in each cluster.

```
import numpy as np
import matplotlib.pyplot as plt

m = 2  # number of clusters
dp = np.array([(1, 3), (2, 5), (6, 8), (7, 9)])
x = dp[:, 0]
y = dp[:, 1]
prob = np.array([(0.8, 0.2), (0.7, 0.3), (0.2, 0.8), (0.1, 0.9)])
print('Datapoints:', dp)
print('Original membership values:', prob)
c_x = np.zeros(m, dtype=float)
c_y = np.zeros(m, dtype=float)
tolerance_val = 0.01

def update(x, y, c_x, c_y, prob, m):
    distances = np.sqrt((x[:, np.newaxis] - c_x) ** 2 + (y[:, np.newaxis] - c_y) ** 2)
    min_distances = distances.min(axis=1).reshape(-1, 1)
    u = 1.0 / np.sum((distances / min_distances) ** (2 / (m - 1)), axis=1)
    return u

old_u = prob
while True:
    num = 0
    for k in range(m):
        num_x = np.sum((prob[:, k] ** m) * x)
        num_y = np.sum((prob[:, k] ** m) * y)
        den = np.sum(prob[:, k] ** m)
        if den != 0:
            c_x[num] = num_x / den
            c_y[num] = num_y / den
            num += 1
    new_u = update(x, y, c_x, c_y, prob, m)
    if old_u is not prob:
        diff = np.linalg.norm(new_u - old_u)
        if diff < tolerance_val:
            break
    old_u = new_u
    prob = np.column_stack((new_u, 1 - new_u))
```

```
print('Final Membership values:', prob)
print('Centroids',c_x,c_y)
plt.scatter(x, y, label='Data Points')
plt.scatter(c_x, c_y, marker='x', color='red', label='Centroids')
plt.title('Datapoints and Centroids')
plt.legend()
plt.grid(True)
plt.show()
```

```
Datapoints: [[1 3]
 [2 5]
 [6 8]
 [7 9]]
Original membership values: [[0.8 0.2]
 [0.7 0.3]
 [0.2 0.8]
 [0.1 0.9]]
Final Membership values: [[0.49898605 0.50101395]
 [0.49663886 0.50336114]
 [0.4977806  0.5022194 ]
 [0.49862483 0.50137517]]
Centroids [4.00119918 3.99879964] [6.24982605 6.25016397]
```



10. Implement and plot different defuzzification methods:

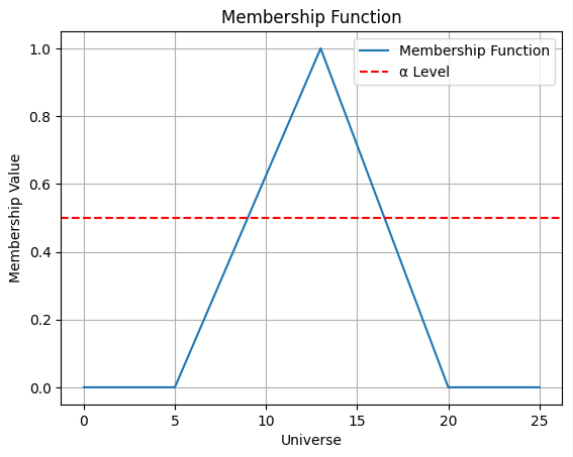
α- cut and strong α- cut:

```
import numpy as np
import matplotlib.pyplot as plt
# Define the membership function manually
def membership_function(x, a, b, c):
    return np.maximum(0, np.minimum((x - a) / (b - a), (c - x) / (c - b)))
# Define the α-cut defuzzification method
def alpha_cut_defuzzification(membership_values, universe, alpha):
    weighted_values = membership_values * (membership_values >= alpha)
    weighted_sum = np.sum(weighted_values * universe)
    total_weight = np.sum(membership_values * (membership_values >= alpha))
    if total_weight == 0:
        return 0 # Avoid division by zero
    return weighted_sum / total_weight
# Define the strong α-cut defuzzification method
def strong_alpha_cut_defuzzification(membership_values, universe, alpha):
    return np.mean(universe[membership_values >= alpha])
# Define universe and membership function parameters
universe = np.arange(0, 26, 1)
a = 5
b = 13
c = 20
# Calculate membership values
membership_values = membership_function(universe, a, b, c)
# Define α level
alpha = 0.5
# Perform α-cut defuzzification
alpha_cut_result = alpha_cut_defuzzification(membership_values, universe,
alpha)
print("α-cut defuzzification result:", alpha_cut_result)
# Perform strong α-cut defuzzification
strong_alpha_cut_result =
strong_alpha_cut_defuzzification(membership_values, universe, alpha)
```

```
print("Strong α-cut defuzzification result:", strong_alpha_cut_result)
# Plot the membership function
plt.plot(universe, membership_values, label='Membership Function')
plt.xlabel('Universe')
plt.ylabel('Membership Value')
plt.title('Membership Function')
plt.axhline(y=alpha, color='r', linestyle='--', label='α Level')
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
α-cut defuzzification result: 12.61818181818182
Strong α-cut defuzzification result: 12.5
```



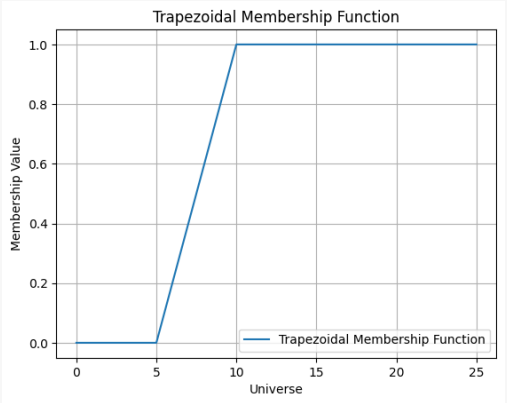
Maxima method:

source code:

```
import numpy as np

import matplotlib.pyplot as plt

# Define the trapezoidal membership function
def trapezoidal_mf(x, a, b, c, d):
    return np.maximum(0, np.minimum((x - a) / (b - a), 1, (d - x) / (d -
c)))
# Define the Maxima methods
def fom_defuzzification(membership_values, universe):
    maxima_indices = np.where(membership_values ==
np.max(membership values))[0]
```

```
    return universe[maxima_indices[0]] if maxima_indices.size > 0 else 0
def lom_defuzzification(membership_values, universe):
    maxima_indices = np.where(membership_values ==
np.max(membership_values))[0]
    return universe[maxima_indices[-1]] if maxima_indices.size > 0 else 0
def mom_defuzzification(membership_values, universe):
    maxima_indices = np.where(membership_values ==
np.max(membership_values))[0]
    return np.mean(universe[maxima_indices]) if maxima_indices.size > 0
else 0
# Define universe and parameters for trapezoidal MF
universe = np.arange(0, 26, 1)
a = 5
b = 10
c = 15
d = 20
# Calculate membership values using trapezoidal MF
membership_values = trapezoidal_mf(universe, a, b, c, d)
# Plot the membership function
plt.plot(universe, membership_values, label='Trapezoidal Membership
Function')
plt.xlabel('Universe')
plt.ylabel('Membership Value')
plt.title('Trapezoidal Membership Function')
plt.legend()
plt.grid(True)
plt.show()
```

Ouput:



a. First of Maxima (FOM)

Source Code:

```
# Perform FOM defuzzification
fom_result = fom_defuzzification(membership_values, universe)
print("FOM defuzzification result:", fom_result)
```

Output:

```
FOM defuzzification result: 10
```

b. Last of Maxima (LOM)

Source Code:

```
# Perform LOM defuzzification
lom_result = lom_defuzzification(membership_values, universe)
print("LOM defuzzification result:", lom_result)
```

Output:

```
LOM defuzzification result: 25
```

c. Mean of Maxima (MOM)

```
# Perform MOM defuzzification
mom_result = mom_defuzzification(membership_values, universe)
print("MOM defuzzification result:", mom_result)
```

Output:

```
MOM defuzzification result: 17.5
```

Centroid method:

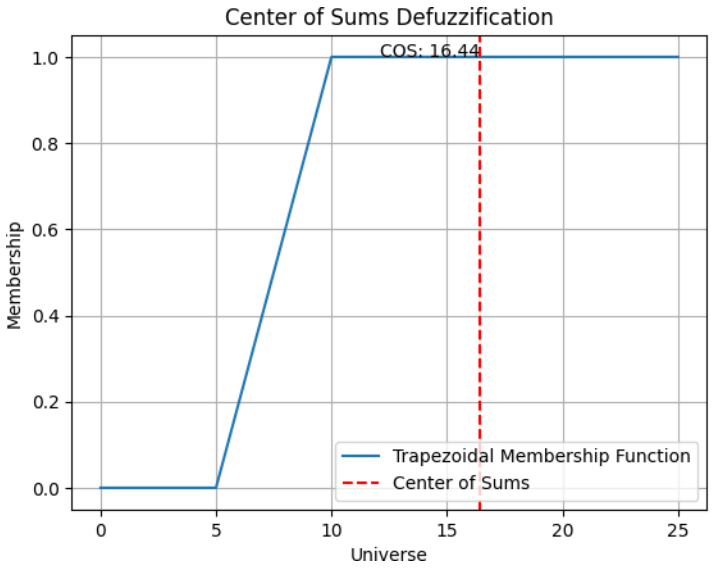
a. Center of Sums (COS) :

```
import numpy as np
import matplotlib.pyplot as plt
# Define the trapezoidal membership function
def trapezoidal_mf(x, a, b, c, d):
    return np.maximum(0, np.minimum((x - a) / (b - a), 1, (d - x) /
(d - c)))
# Define the Center of Sums (COS) defuzzification method
def center_of_sums(memberships, universe):
    return np.sum(memberships * universe) / np.sum(memberships)
# Define universe and parameters for trapezoidal MF
universe = np.arange(0, 26, 1)
```

```
a = 5
b = 10
c = 15
d = 20
# Calculate membership values using trapezoidal MF
membership_values = trapezoidal_mf(universe, a, b, c, d)
# Perform COS defuzzification
cos_result = center_of_sums(membership_values, universe)
print("COS defuzzification result:", cos_result)
# Plot the membership function and COS
plt.plot(universe, membership_values, label='Trapezoidal Membership
Function')
plt.axvline(x=cos_result, color='r', linestyle='--', label='Center
of Sums')
plt.text(cos_result, max(membership_values), f'COS:
{cos_result:.2f}', ha='right')
plt.xlabel('Universe')
plt.ylabel('Membership')
plt.title('Center of Sums Defuzzification')
plt.legend()
plt.grid(True)
plt.show()
```

Output:

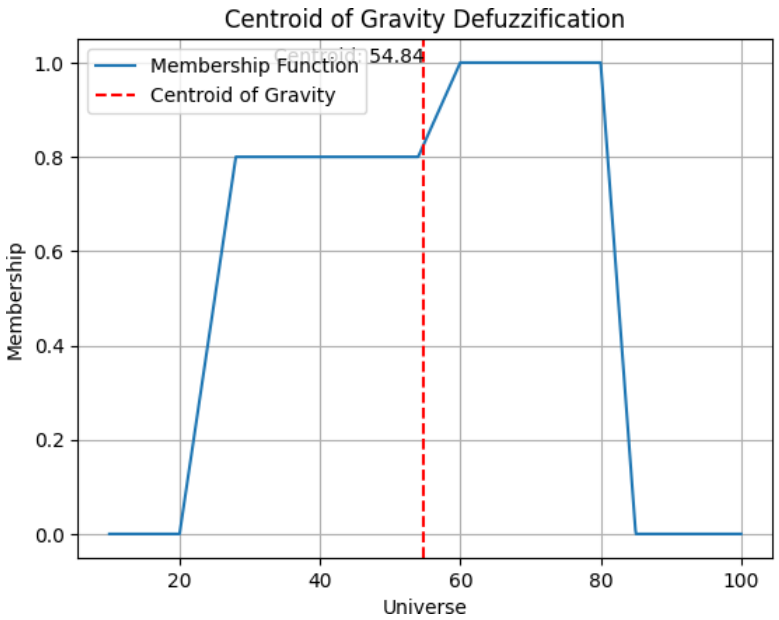
```
COS defuzzification result: 16.444444444444443
```



a. Center of gravity (COG) / Centroid of Area (COA) Method

```
import matplotlib.pyplot as plt

def plot_centroid_of_gravity(memberships, universe):
    plt.plot(universe, memberships, label='Membership Function')
    centroid = centroid_of_gravity(memberships, universe)
    if centroid is not None:
        plt.axvline(x=centroid, color='r', linestyle='--', label='Centroid
of Gravity')
        plt.text(centroid, max(memberships), f'Centroid: {centroid:.2f}',
ha='right')

    plt.xlabel('Universe')
    plt.ylabel('Membership')
    plt.title('Centroid of Gravity Defuzzification')
    plt.legend()
    plt.grid(True)
    plt.show()

plot_centroid_of_gravity(u_val,x)
```

Output:



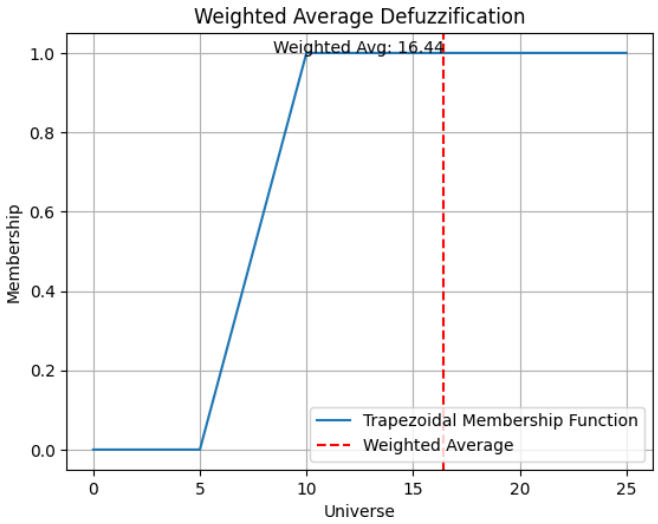
b. Weighted average method

```
import numpy as np
import matplotlib.pyplot as plt
# Define the trapezoidal membership function
def trapezoidal_mf(x, a, b, c, d):
    return np.maximum(0, np.minimum((x - a) / (b - a), 1, (d - x) /
(d - c)))
# Define the Weighted Average defuzzification method
def weighted_average_defuzzification(memberships, universe):
    return np.average(universe, weights=memberships)
# Define universe and parameters for trapezoidal MF
universe = np.arange(0, 26, 1)
a = 5
b = 10
c = 15
d = 20
# Calculate membership values using trapezoidal MF
membership_values = trapezoidal_mf(universe, a, b, c, d)
# Perform Weighted Average defuzzification
weighted_avg_result =
weighted_average_defuzzification(membership_values, universe)
print("Weighted Average defuzzification result:",
weighted_avg_result)
# Plot the membership function and weighted average
plt.plot(universe, membership_values, label='Trapezoidal Membership
Function')
plt.axvline(x=weighted_avg_result, color='r', linestyle='--',
label='Weighted Average')
plt.text(weighted_avg_result, max(membership_values), f'Weighted
Avg: {weighted_avg_result:.2f}', ha='right')

plt.xlabel('Universe')
plt.ylabel('Membership')
plt.title('Weighted Average Defuzzification')
plt.legend()
plt.grid(True)
```

```
plt.show()
```

Output:

```
Weighted Average defuzzification result: 16.444444444444443
```



11. Implement the AND, OR and XOR function using single layer and multi-layer neural network.

a.. Implement the AND, OR using single layer neural network.

```
import numpy as np
import matplotlib.pyplot as plt
def perceptron_train(inputs, labels, learning_rate=0.1,
epochs=4):
    num_inputs = inputs.shape[1]
    weights = np.zeros(num_inputs + 1) # Initialize weights
(including bias)

    print("Initial Weights:", weights)

    for   epoch   in   range(epochs):
        print("\nEpoch:", epoch + 1)
        for input_row, label in zip(inputs, labels):
            prediction = predict(input_row, weights)
            error = label - prediction
            weights[1:] += learning_rate * error * input_row
```

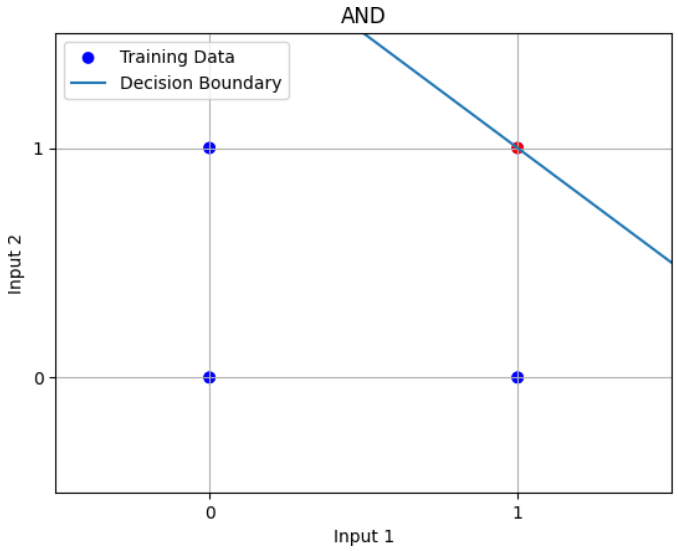
```
            weights[0] += learning_rate * error # Update bias
            print("Weights after training sample:", weights)

    return weights

def predict(inputs, weights):
    summation = np.dot(inputs, weights[1:]) + weights[0]
    return 1 if summation >= 0 else 0
def plot_decision_boundary(inputs, labels, weights, title):
    plt.figure()
    plt.title(title)
    plt.xlabel("Input 1")
    plt.ylabel("Input 2")
    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)
    plt.xticks([0, 1])
    plt.yticks([0, 1])

    plt.scatter(inputs[:, 0], inputs[:, 1], c=labels,
cmap='bwr', label='Training Data')

    x_values = np.linspace(-0.5, 1.5)
    y_values = -(weights[1] * x_values + weights[0]) /
weights[2]
    plt.plot(x_values, y_values, label='Decision Boundary')

    plt.legend()
    plt.grid(True)
    plt.show()
def logical_and():
    return np.array([[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1,
1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]]), np.array([0,
0, 0, 0, 0, 0, 0, 1])

def logical_or():
```

```
    return np.array([[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]]), np.array([0, 1, 1, 1, 1, 1, 1, 1])

# Train and plot for each logical operation

operations = {
    "AND": logical_and,
    "OR": logical_or,
}

for operation, func in operations.items():
    inputs, labels = func()
    weights = perceptron_train(inputs, labels)
    plot_decision_boundary(inputs, labels, weights, operation)
```

Output:

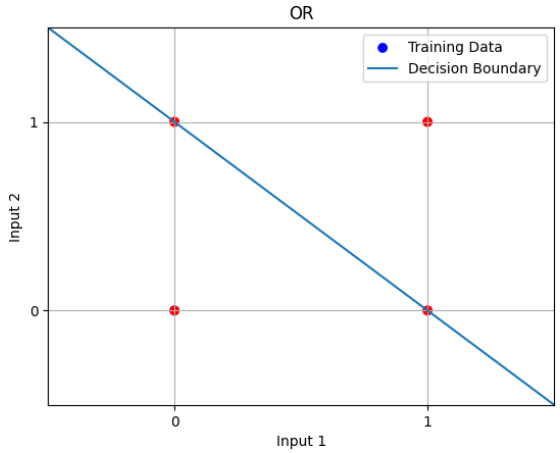
```
Initial Weights: [0. 0. 0. 0.]

Epoch: 1
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [0.  0.1 0.1 0.1]

Epoch: 2
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0. ]
Weights after training sample: [-0.2  0.1  0.1  0. ]
Weights after training sample: [-0.2  0.1  0.1  0. ]
Weights after training sample: [-0.2  0.1  0.1  0. ]
Weights after training sample: [-0.2  0.1  0.1  0. ]
Weights after training sample: [-0.3  0.   0.   0. ]
Weights after training sample: [-0.2  0.1  0.1  0.1]

Epoch: 3
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
```

```
Epoch: 4
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
Weights after training sample: [-0.2  0.1  0.1  0.1]
```




```
Initial Weights: [0. 0. 0. 0.]

Epoch: 1
Weights after training sample: [-0.1  0.   0.   0. ]
Weights after training sample: [0.  0.  0.  0.1]
Weights after training sample: [0.  0.  0.  0.1]
Weights after training sample: [0.  0.  0.  0.1]
Weights after training sample: [0.  0.  0.  0.1]
Weights after training sample: [0.  0.  0.  0.1]
Weights after training sample: [0.  0.  0.  0.1]
Weights after training sample: [0.  0.  0.  0.1]

Epoch: 2
Weights after training sample: [-0.1  0.   0.   0.1]
Weights after training sample: [-0.1  0.   0.   0.1]
Weights after training sample: [0.  0.  0.1 0.1]
Weights after training sample: [0.  0.  0.1 0.1]
Weights after training sample: [0.  0.  0.1 0.1]
Weights after training sample: [0.  0.  0.1 0.1]
Weights after training sample: [0.  0.  0.1 0.1]
Weights after training sample: [0.  0.  0.1 0.1]

Epoch: 3
Weights after training sample: [-0.1  0.   0.1  0.1]
Weights after training sample: [-0.1  0.   0.1  0.1]
Weights after training sample: [-0.1  0.   0.1  0.1]
Weights after training sample: [-0.1  0.   0.1  0.1]
```

```
Weights after training sample: [0.  0.1 0.1 0.1]
Weights after training sample: [0.  0.1 0.1 0.1]
Weights after training sample: [0.  0.1 0.1 0.1]
Weights after training sample: [0.  0.1 0.1 0.1]

Epoch: 4
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
Weights after training sample: [-0.1  0.1  0.1  0.1]
```



Implement XOR function using multi-layer neural network.

```
import numpy as np
import matplotlib.pyplot as plt
# Define sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
# Define derivative of sigmoid function
def sigmoid_derivative(x):
    return x * (1 - x)
# XOR input and output
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
Y = np.array([[0], [1], [1], [0]])
# Hyperparameters
input_size = 2
```

```
hidden_size = 4
output_size = 1
learning_rate = 0.1
epochs = 10000
# Initialize weights and biases randomly
weights_input_hidden = np.random.uniform(size=(input_size,
hidden_size))
biases_hidden = np.random.uniform(size=(1, hidden_size))
weights_hidden_output = np.random.uniform(size=(hidden_size,
output_size))
biases_output = np.random.uniform(size=(1, output_size))
# Training loop
losses = []
for epoch in range(epochs):
    # Forward propagation
    hidden_input = np.dot(X, weights_input_hidden) + biases_hidden
    hidden_output = sigmoid(hidden_input)
    output = np.dot(hidden_output, weights_hidden_output) +
biases_output
    predicted_output = sigmoid(output)
    # Backpropagation
    output_error = Y - predicted_output
    output_delta = output_error *
sigmoid_derivative(predicted_output)

    hidden_error = output_delta.dot(weights_hidden_output.T)
    hidden_delta = hidden_error * sigmoid_derivative(hidden_output)
    # Update weights and biases
    weights_hidden_output += hidden_output.T.dot(output_delta) *
learning_rate
    biases_output += np.sum(output_delta, axis=0, keepdims=True) *
learning_rate
    weights_input_hidden += X.T.dot(hidden_delta) * learning_rate
    biases_hidden += np.sum(hidden_delta, axis=0, keepdims=True) *
learning_rate
    # Calculate and store loss
    loss = np.mean(np.square(output_error))
```

```
losses.append(loss)
# Define grid points for decision boundary

xx, yy = np.meshgrid(np.linspace(0, 1, 100), np.linspace(0, 1, 100))

grid_points = np.c_[xx.ravel(), yy.ravel()]
# Plot decision boundary
plt.contourf(xx, yy, predicted_classes.reshape(xx.shape), alpha=0.3)
plt.scatter(X[:, 0], X[:, 1], c=Y.flatten(), cmap='viridis')
plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.title('XOR Decision Boundary')
plt.show()
```

Output: